DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Zargux

Github Link: https://github.com/abhyuday-fr/Zargux

Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY: Abhyuday Pundir (Roll No: 2400320100059) Degree: Bachelor of Technology (B.Tech) Branch: Computer Science & Engineering / Cyber Security Academic Year: 2026 - 2027	UNDER THE GUIDANCE OF: Mr. Sarath G. Status: 3rd Year Project Specification Date of Submission: 01-10-2026

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	30-09-2026	Initial Draft & Scope Definition	Student	Project Trainer
1.0	01-10-2026	Final Day-4/Day-5 SRS Baseline	Student	Project Trainer

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functionality, interfaces, constraints, and verification criteria for Zargux, a Linux server incident detection prototype. It provides a common reference for the student developer, project trainer, and academic evaluators.

1.2 Scope of the System

Zargux monitors Linux CPU pressure using Pressure Stall Information (PSI), inspects running process executable paths through /proc/<pid>/exe, checks Debian package ownership using dpkg, and classifies process origin into CLEAR, WATCH, or HIGH verdicts. The prototype focuses on identifying processes running from writable temporary locations or deleted executables while providing CPU pressure and process CPU information.

In-Scope Capabilities: Controlled stress-ng simulations, Ubuntu VM execution through Multipass, CPU PSI monitoring, per-process executable-origin inspection, dpkg ownership checks, process CPU usage display, severity classification, findings persistence in JSON, one-shot scanning, continuous watch mode, and a reproducible three-case demonstration.

Out-of-Scope Boundaries: Production hardening, real malware analysis, attacks against unauthorised systems, multi-tenant deployment, automated process termination or isolation, and use of real sensitive operational data.

Expected Benefits: Demonstrates a reproducible Linux incident detection workflow in a controlled lab environment and provides evidence through CLI output and persisted findings.

1.3 Definitions, Acronyms, and Abbreviations

Category	Term / Acronym	Definition / Standard Context
Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Security	IDS	Intrusion Detection System - software that flags suspicious activity.
Security	FN / FP	False Negative / False Positive - a missed or wrongly-raised alert.
Architecture	API	Application Programming Interface for data interchange via JSON/HTTP(S).
Project Specific	PSI	Pressure Stall Information. Linux kernel interface used by Zargux to read CPU pressure metrics.

1.4 References

[1] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

[2] The Linux Kernel Documentation, “Pressure Stall Information (PSI),” https://docs.kernel.org/accounting/psi.html

[3] dpkg(1) manual page, Debian Project, https://man7.org/linux/man-pages/man1/dpkg.1.html

[4] systemd-run(1) manual page, freedesktop.org, https://www.freedesktop.org/software/systemd/man/systemd-run.html

[5] Zig Software Foundation, “Zig 0.13.0 Documentation,”https://ziglang.org/documentation/0.13.0/

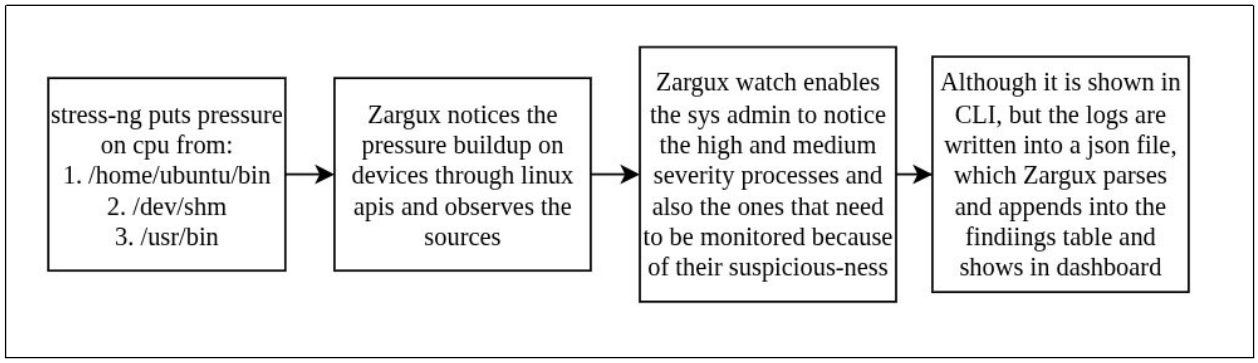
1.5 Document Overview

The remainder of this SRS describes the product perspective, user classes, operating environment, constraints, functional requirements, external interfaces, non-functional requirements, system models, and academic sign-off.

2. Overall Description

2.1 Product Perspective

Zargux is a local Linux server incident detection prototype. Multipass provides an Ubuntu VM used as the controlled target environment. The prototype is implemented in Python and runs inside the VM. It reads CPU PSI from /proc/pressure/cpu, enumerates processes through /proc, reads each process executable path from /proc/<pid>/exe, checks whether the executable is owned by a Debian package using dpkg -S, and classifies the process origin. CLEAR processes are package-managed, WATCH processes are unpackaged but not located in the monitored temporary directories, and HIGH processes are deleted executables or executables running from /dev/shm, /tmp, or /var/tmp. The watch mode continuously refreshes the CLI and records new findings in findings.json.



2.2 User Classes and Characteristics

The target audience for this tool during the demo:

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights

Operator (you)	Intermediate	Runs the controlled demo, scan, watch, and demo commands, and reviews findings.json and CLI output.
Evaluator / Viewer	Basic to Intermediate	Read-only: observes the CLI output, validation cases, and saved findings.

2.3 Operating Environment

Client Requirements: A Linux terminal. No web browser is required because the current prototype provides a CLI interface rather than a web dashboard.

Runtime Environment: Ubuntu Linux VM managed through Multipass, Python 3.x, procfs, dpkg, systemd, systemctl, ps, and stress-ng for the controlled demonstration.

Storage: JSON file, findings.json, containing the most recent 50 findings.

2.4 Design and Implementation Constraints

Lab-Safety Constraint: All simulations are controlled demonstrations on the Ubuntu VM. No real malware, real credentials, or unauthorised systems are used.

Time Constraint: The current SRS describes the prototype scope that can be demonstrated reproducibly in the laboratory environment.

Tooling Constraint: The prototype uses Python and standard Linux utilities available in the Ubuntu VM, with stress-ng and systemd-run used for controlled test execution.

2.5 Assumptions and Dependencies

Assumptions: The Ubuntu VM has procfs access and the required Linux utilities installed. sudo privileges are available for the demo because systemd-run is used to create controlled processes and /proc entries of other users may need to be read.

Dependencies: Python 3.x, Linux procfs, dpkg, ps, systemd-run, systemctl, stress-ng, and standard Python modules including argparse, json, os, subprocess, datetime, shutil, sys, and time.

3. System Features and Functional Requirements

The functional requirements are grouped into simulation, detection, and findings/response modules. Each requirement has a unique identifier and is limited to behaviour implemented by the current prototype.

3.1 Module 1: Attack / Simulation Component

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	Launch the controlled legitimate stress-ng	The Ubuntu VM has stress-ng installed and	High

	process from /usr/bin.	the process is launched with systemd-run.	
FR-1.2	Create a dropped-copy test by copying stress-ng to /dev/shm and executing the copy.	The copied executable exists at /dev/shm/stress-copy and is executable.	High
FR-1.3	Run a negative control using a legitimate but unpackaged executable from ~/bin.	The copied /bin/sleep executable runs from /home/ubuntu/bin/my-legit-tool.	Medium

3.2 Module 2: Detection Engine

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Read CPU pressure from /proc/pressure/cpu and inspect running process executable paths through /proc/<pid>/exe.	The required procfs entries are readable. Unavailable or exited processes are skipped, while scan reports an error when its target cannot be read.	High
FR-2.2	Classify each executable origin using deleted-file detection, monitored temporary-path checks, and dpkg ownership.	Classification returns CLEAR, WATCH, or HIGH with a reason string.	High
FR-2.3	Display process CPU usage and CPU PSI values during continuous watch mode.	CPU usage is read with ps and PSI values are read from /proc/pressure/cpu.	Medium

3.3 Module 3: Response & Dashboard

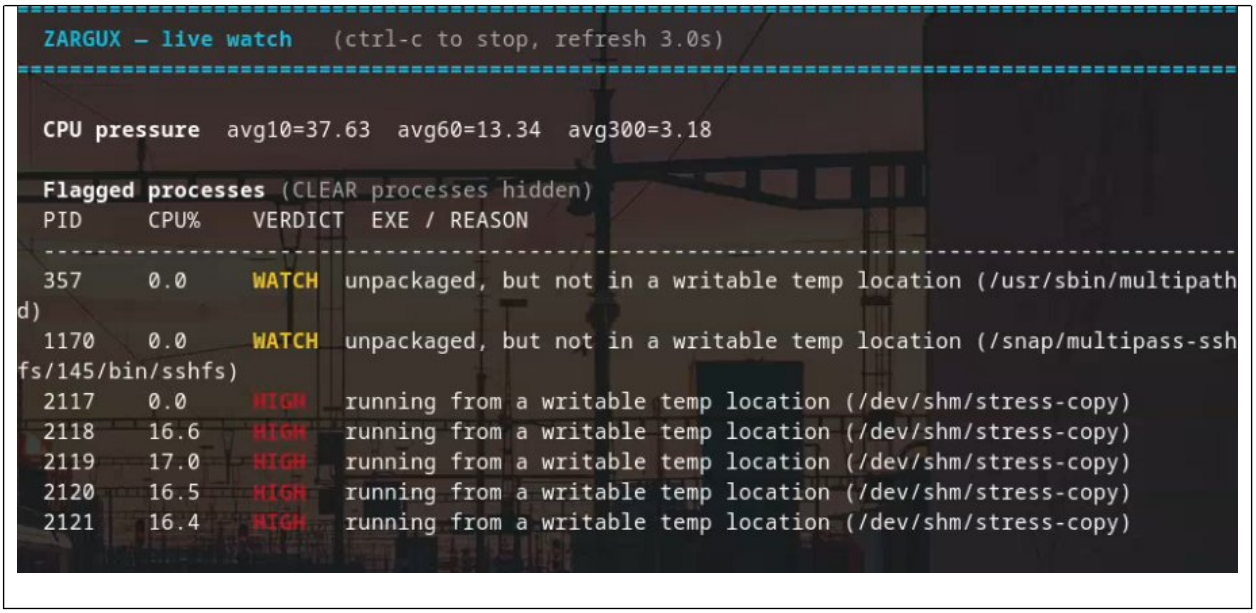
Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Record findings with time, summary, and severity in findings.json.	The JSON file is updated and limited to the latest 50 findings.	High
FR-3.2	Generate a severity record for each detected finding.	Severity mapping is HIGH to High, WATCH to Medium, and CLEAR to Info.	High
FR-3.3	Provide scan, watch, and demo CLI views for operators and evaluators.	scan requires a PID; watch supports an optional refresh interval; demo executes the three	Medium

		validation cases.	

4. External Interface Requirements

4.1 User Interfaces (UI)

The current user interface is a terminal CLI. The scan command shows the PID, executable path, verdict, and reason. The watch command displays CPU PSI averages, CPU usage, and all non-CLEAR processes. Verdicts are colour-coded when the terminal supports ANSI colours. The demo command runs three controlled cases and prints a comparison table.



4.2 Hardware Interfaces

Standard host computer, display, keyboard, and mouse. The detector itself runs inside the Ubuntu VM provided by Multipass. No specialised hardware is required.

4.3 Software Interfaces

Python standard library modules: argparse, datetime, json, os, shutil, subprocess, sys, and time. External Linux tools: stress-ng, dpkg, ps, systemd-run, systemctl, and procfs (/proc).

4.4 Communication Interfaces

None. The current prototype is a local CLI application running inside the Ubuntu VM. Findings are persisted to a local JSON file.

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification & Testing Method
5.1 Performance	Watch refreshes at the configured interval, default 3	Run zargux watch and verify repeated refreshes at the

	seconds. A scan completes after reading the target process and applying the classification rules.	configured interval. Run zargux scan against a live PID and confirm it exits cleanly.
5.2 Security	The prototype is intended for controlled lab use and requires appropriate permissions to inspect process executable paths and run the demo.	Run only inside the Ubuntu VM. Verify that no real credentials or malware are used. Confirm that sudo is used only where required by the demonstration.
5.3 Reliability	Watch mode continues refreshing until interrupted and avoids repeated logging of the same PID at the same verdict.	Run zargux watch through the demonstration period and confirm no crash. Check findings.json for non-duplicated refresh logging.
5.4 Accuracy	The three-case demo should distinguish the package-managed /usr/bin/stress-ng, the /dev/shm dropped copy, and the legitimate unpackaged ~/bin control.	Run zargux demo and compare the three verdicts side by side. The /dev/shm copy must be HIGH and the /usr/bin copy must be CLEAR.

6. System Design and Analysis Models

Include the following standard diagrams, matching your own project:

Diagram Type	Objective & Scope	Deliverable Format
Use Case Diagram	Shows the Operator running scan, watch, and demo, while the Evaluator / Viewer observes CLI output and findings.	[INSERT FIGURE 2 HERE: UML Use Case diagram]
Data Flow Diagram (DFD)	Shows the flow from CPU PSI and process data sources to classification, findings.json, and CLI output.	[INSERT FIGURE 3 HERE: Level 0 and Level 1 DFD]
Entity-Relationship (ERD)	The prototype uses JSON rather than a relational database. A finding contains time, summary, and severity.	[INSERT FIGURE 4 HERE: findings.json data model, or write Not applicable for relational ERD]

7. Academic Review & Sign-Off

This Software Requirements Specification has been submitted and reviewed:

Project Trainer / Guide	Student
Signature: _____ Date: _____	Signature: _____ Date: _____